Red Social para Mascotas con Flutter Web + Firebase.

Proyecto: "PetVerse - El universo donde tus mascotas son las estrellas".

Tecnologías: Flutter Web, Dart, Firebase (Auth, Firestore, Storage, Cloud Functions, Cloud Messaging), Provider, PWA.

Objetivo general: Construir una aplicación web progresiva (PWA) tipo red social para mascotas, con autenticación, feed de publicaciones, sistema de adopción, mensajería y notificaciones push, aplicando buenas prácticas de desarrollo y arquitectura serverless.

Introducción – El universo de las mascotas digitales.

El fenómeno pet-tech.

Las mascotas ya no son solo compañeras: son celebrities digitales. Cuentas de Instagram como `@jiffpom`, `@nala_cat` o `@tunameltsmyheart` tienen millones de seguidores. El mercado pet-tech mueve más de 300 mil millones de dólares al año y sigue creciendo. Las apps como Rover, Wag!, Petfinder y BarkBox demuestran que hay un negocio real detrás de las mascotas.

PetVerse nace como una plataforma donde las mascotas son las protagonistas:

- Creadores de contenido publican fotos y videos de sus perros y gatos.
- Amantes de mascotas descubren, siguen y reaccionan al contenido.
- Refugios y rescatistas publican mascotas en adopción.
- La comunidad se conecta mediante likes, comentarios y mensajes.

¿Por qué Flutter Web + Firebase?

Flutter Web permite construir interfaces hermosas y consistentes con un solo código

Dart que corre en el navegador. Además, al compilarse a PWA, se puede instalar como app nativa en el celular.

Firebase es el backend serverless por excelencia:

- **Authentication:** Login con email, Google, Apple.
- **Firestore:** Base de datos NoSQL en tiempo real.
- **Storage:** Almacenamiento de fotos y videos.
- **Cloud Functions:** Lógica backend sin servidores.
- **Cloud Messaging:** Notificaciones push.

¿Qué aprenderán?

- **Flutter Web:** Responsive design, PWA, widgets, animaciones.
- **Firebase Auth:** Flujos de autenticación modernos.
- **Firestore:** Queries en tiempo real, subcolecciones, índices.
- **Storage:** Subida de imágenes/videos con progreso.
- **Cloud Functions:** Funciones trigger y callables.
- **State Management:** Provider / Riverpod para estado global.
- **Serverless:** Arquitectura sin servidor, escalable, sin DevOps.

Este proyecto es parte del “Sistema Integrador del segundo cuatrimestre” y deberá ser defendido mostrando cada decisión de código.

Flutter Web y PWA.

Flutter es el framework de Google para construir interfaces nativas multiplataforma con un solo código Dart. Flutter Web compila a JavaScript y HTML, permitiendo que la app corra en cualquier navegador.

Particularidades de Flutter Web:

- **CanvasKit vs HTML Renderer:** CanvasKit usa WebAssembly (más rápido, más pesado); HTML usa el DOM (más compatible, más liviano).
- **Responsive Design:** Usar `LayoutBuilder`, `MediaQuery` y `Flexible` para adaptarse a diferentes pantallas.
- **PWA:** Agregar `manifest.json` y `service worker` para que la app sea instalable.

Configuración de PWA en `web/index.html`:

HTML:

```
<link rel="manifest" href="manifest.json">
<meta name="theme-color" content="#FF6B35">
```

`web/manifest.json`:

```
{
  "name": "PetVerse",
  "short_name": "PetVerse",
  "start_url": ".",
  "display": "standalone",
  "background_color": "#FFFFFF",
  "theme_color": "#FF6B35",
  "description": "Red social para mascotas",
  "orientation": "portrait-primary",
  "icons": [
    { "src": "icons/Icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icons/Icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

Compilar para web:

```
flutter build web --release
```

```
# Desplegar la carpeta build/web en Vercel/Netlify
```

Firebase Authentication.

Firebase Auth provee autenticación lista para usar con múltiples proveedores.

Configuración inicial (`lib/main.dart`):

```
await Firebase.initializeApp(
  options: const FirebaseOptions(
    apiKey: 'TU_API_KEY',
    appId: 'TU_APP_ID',
    messagingSenderId: 'TU_SENDER_ID',
    projectId: 'petverse-2026',
    authDomain: 'petverse-2026.firebaseapp.com',
    storageBucket: 'petverse-2026.appspot.com',
  ),
);
```

Login con email:

```
Future<UserCredential> signInWithEmail(String email, String password) async {
  return await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

Login con Google:

```
Future<UserCredential> signInWithGoogle() async {
final GoogleAuthProvider googleProvider = GoogleAuthProvider();
```

```
  return await FirebaseAuth.instance.signInWithPopup(googleProvider);
}
```

Escuchar cambios de sesión:

```
FirebaseAuth.instance.authStateChanges().listen((User? user) {
  if (user == null) {
    print('Usuario desconectado');
  } else {
    print('Usuario: ${user.email}');
  }
});
```

Firestore – Base de Datos en Tiempo Real.

Firestore es una base de datos NoSQL orientada a documentos con sincronización en tiempo real.

Estructura de colecciones en PetVerse:

```
users/{userId}
  ├── name, email, photoUrl, bio, createdAt
  └── pets/{petId}
      └── name, type, breed, age, photos[], bio

posts/{postId}
  ├── petId, ownerId, imageUrl, caption, likes[], createdAt
  └── comments/{commentId}
      └── userId, userName, text, createdAt

adoptions/{petId}
  └── (datos de la mascota + info de adopción)
```

```
chats/{chatId}
  └── messages/{messageId}
      └── senderId, text, timestamp
```

Operaciones básicas:

Crear documento:

```
await FirebaseFirestore.instance.collection('posts').add({
  'petId': petId,
  'ownerId': userId,
  'imageUrl': imageUrl,
  'caption': caption,
  'likes': [],
  'createdAt': FieldValue.serverTimestamp(),
});
```

Leer con stream (tiempo real):

```
StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
      .collection('posts')
      .orderBy('createdAt', descending: true)
      .limit(20)
      .snapshots(),
  builder: (context, snapshot) { ... },
);
```

Actualizar con operaciones atómicas:

```
await postRef.update({
'likes': FieldValue.arrayUnion([userId]),
```

```
});
```

Índices compuestos: Firestore requiere índices para queries con múltiples `where` + `orderBy`. Se configuran en la consola de Firebase.

Firebase Storage – Subida de Fotos/Videos.

Firebase Storage almacena archivos (imágenes, videos, audios).

Subida con progreso:

```
Future<String> uploadImage(File file, String userId) async {
  final ref = FirebaseStorage.instance
      .ref()
      .child('users/$userId/posts/${DateTime.now().millisecondsSinceEpoch}.jpg');

  final uploadTask = ref.putFile(file);

  uploadTask.snapshotEvents.listen((TaskSnapshot snapshot) {
    final progress = snapshot.bytesTransferred / snapshot.totalBytes;
    print('Progreso: ${(progress * 100).toStringAsFixed(1)}%');
  });

  final snapshot = await uploadTask;
  return await snapshot.ref.getDownloadURL();
}
```

En Flutter Web, `File` no funciona como en mobile. Se debe usar `Uint8List`:

```
final bytes = await file.readAsBytes();
final ref = FirebaseStorage.instance.ref().child('posts/image.jpg');
await ref.putData(bytes);
final url = await ref.getDownloadURL();
```

Cloud Functions – Backend sin Servidor.

Cloud Functions ejecuta código Node.js en respuesta a eventos (creación de documento, HTTP request, etc.).

Tipos de funciones:

- **Trigger functions:** Reaccionan a eventos de Firestore, Auth, Storage.
- **Callable functions:** Invocables desde el cliente con `httpsCallable`.
- **HTTP functions:** Endpoints REST.

Ejemplo – Notificación al recibir un like:

```
// functions/index.js
const functions = require('firebase-functions');
const admin = require('firebase-admin');
admin.initializeApp();

exports.onNewLike = functions.firestore
  .document('posts/{postId}')
  .onUpdate(async (change, context) => {
    const before = change.before.data();
    const after = change.after.data();

    if (after.likes.length > before.likes.length) {
      const newLiker = after.likes[after.likes.length - 1];
      const ownerId = after.ownerId;

      if (newLiker !== ownerId) {
        const ownerDoc = await admin.firestore()
          .collection('users').doc(ownerId).get();
        const fcmToken = ownerDoc.data()?.fcmToken;
```

```
    if (fcmToken) {
      await admin.messaging().send({
        token: fcmToken,
        notification: {
          title: '¡Nuevo like!',
          body: 'A alguien le gustó tu publicación',
        },
      });
    }
  }
}
});
```

Deploy:

```
firebase deploy --only functions
```

Cloud Messaging – Notificaciones Push.

Firebase Cloud Messaging (FCM) permite enviar notificaciones push a los usuarios.

En Flutter:

```
final fcmToken = await FirebaseMessaging.instance.getToken();
await FirebaseFirestore.instance.collection('users').doc(userId).update({
  'fcmToken': fcmToken,
});
```

En Flutter Web: Requiere configurar un Service Worker (`firebase-messaging-sw.js`) en la carpeta `web/`.

State Management con Provider.

Provider es la solución oficial recomendada por el equipo de Flutter para state management simple.

Ejemplo de AuthProvider:

```
class AuthProvider extends ChangeNotifier {
  User? _user;
  User? get user => _user;

  AuthProvider() {
    FirebaseAuth.instance.authStateChanges().listen((user) {
      _user = user;
      notifyListeners();
    });
  }

  Future<void> signIn(String email, String password) async {
    await FirebaseAuth.instance.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
  }

  Future<void> signOut() async {
    await FirebaseAuth.instance.signOut();
  }
}
```

En `main.dart`:

```
MultiProvider(
providers: [
```

```
    ChangeNotifierProvider(create: (_) => AuthProvider()),
    ChangeNotifierProvider(create: (_) => PetProvider()),
  ],
  child: MaterialApp(...),
);
```

Despliegue en Vercel/Netlify.

Flutter Web genera archivos estáticos en `build/web/`. Se puede desplegar fácilmente:

Vercel:

```
npm install -g vercel
cd build/web
vercel --prod
```

Netlify:

```
npm install -g netlify-cli
cd build/web
netlify deploy --prod --dir=.
```

Firebase Hosting:

```
firebase init hosting
firebase deploy --only hosting
```

Ejemplo: "Blog de Viajes Minimalista con Firebase".

Para que vean cómo se estructura una app Flutter Web + Firebase, construyamos un mini blog de viajes.

Estructura:

```
lib/
├── main.dart
├── models/
│   └── post.dart
├── screens/
│   ├── login_screen.dart
│   ├── home_screen.dart
│   └── new_post_screen.dart
├── services/
│   ├── auth_service.dart
│   └── post_service.dart
└── widgets/
    └── post_card.dart
```

Modelo (`lib/models/post.dart`):

```
class Post {
  final String id;
  final String title;
  final String content;
  final String imageUrl;
  final String authorId;
  final DateTime createdAt;

  Post({
    required this.id,
    required this.title,
    required this.content,
    required this.imageUrl,
    required this.authorId,
```

```
    required this.createdAt,
  });

  Map<String, dynamic> toMap() => {
    'title': title,
    'content': content,
    'imageUrl': imageUrl,
    'authorId': authorId,
    'createdAt': createdAt.toIso8601String(),
  };

  factory Post.fromMap(String id, Map<String, dynamic> map) => Post(
    id: id,
    title: map['title'],
    content: map['content'],
    imageUrl: map['imageUrl'],
    authorId: map['authorId'],
    createdAt: DateTime.parse(map['createdAt']),
  );
}
```

Servicio (`lib/services/post_service.dart`):

```
class PostService {
  final _collection = FirebaseFirestore.instance.collection('posts');

  Future<void> createPost(Post post) async {
    await _collection.add(post.toMap());
  }

  Stream<List<Post>> getPosts() {
    return _collection
```

```
      .orderBy('createdAt', descending: true)
      .snapshots()
      .map((snapshot) => snapshot.docs
        .map((doc) => Post.fromMap(doc.id, doc.data()))
        .toList());
 }
}
```

UI (`lib/screens/home_screen.dart`):

```
class HomeScreen extends StatelessWidget {
 final PostService _postService = PostService();

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Mi Blog de Viajes')),
   body: StreamBuilder<List<Post>>(
    stream: _postService.getPosts(),
    builder: (context, snapshot) {
     if (!snapshot.hasData) return const Center(child: CircularProgressIndicator());
     final posts = snapshot.data!;
     return ListView.builder(
      itemCount: posts.length,
      itemBuilder: (context, i) => PostCard(post: posts[i]),
     );
    },
   ),
   floatingActionButton: FloatingActionButton(
    onPressed: () => Navigator.pushNamed(context, '/new-post'),
    child: const Icon(Icons.add),
   ),
```

```
    );
  }
}
```

Explicación: Este ejemplo demuestra el flujo completo: modelo → servicio → stream en tiempo real → UI. En PetVerse aplicarán el mismo patrón pero con más entidades (users, pets, posts, comments, chats).

Proyecto PetVerse – Descripción y Requisitos.

Descripción General.

PetVerse es una PWA construida con Flutter Web que funciona como red social para mascotas, con:

- Registro de usuarios y mascotas.
- Feed de publicaciones (fotos y videos).
- Sistema de likes, comentarios y guardados.
- Sección de adopción con filtros.
- Mensajería entre usuarios.
- Notificaciones push.
- Gamificación (logros, niveles, insignias).

Requisitos Funcionales (ID).

ID	Requerimiento	Descripción
PV1	Autenticación	Login/registro con email, Google y/o Apple. Recuperación de contraseña
PV2	Perfil de usuario	Foto, nombre, bio, lista de mascotas. Editable
PV3	Registro de mascotas	Nombre, tipo (perro/gato), raza, edad, peso, fotos, bio

PV4	Feed de publicaciones	Publicaciones con imagen/video, caption, autor, likes, comentarios. Ordenadas por fecha
PV5	Sistema de likes	Doble tap o botón. Animación al dar like. Contador en tiempo real
PV6	Comentarios	Agregar, ver y eliminar comentarios. Tiempo real
PV7	Sección de adopción	Grid de mascotas en adopción con filtros por tipo, edad, ubicación. Detalle con formulario de contacto
PV8	Subida de fotos/videos	Subir imágenes o videos desde el dispositivo a Firebase Storage con progreso
PV9	Mensajería	Chat 1 a 1 entre usuarios, con historial en Firestore
PV10	Notificaciones push	Recibir notificaciones cuando alguien da like, comenta o envía mensaje
PV11	Gamificación	Sistema de puntos por publicar, dar like, comentar. Insignias desbloqueables
PV12	PWA instalable	Manifest, service worker, íconos, install prompt
PV13	Responsive	Adaptable a móvil, tablet y desktop
PV14	Despliegue	Publicada en Vercel/Netlify/Firebase Hosting

Objetivos del Proyecto.

- **Técnico:** Dominar Flutter Web, Firebase y arquitectura serverless.
- **Arquitectónico:** Separar modelos, servicios, providers y UI.
- **Práctico:** Manejar autenticación, storage, tiempo real y notificaciones.
- **Profesional:** Publicar una PWA instalable con backend real.

Comandos clave:

```
flutter create petverse
```

```
cd petverse
flutter config --enable-web
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage provider
image_picker intl
flutter pub add firebase_messaging flutter_local_notifications
dart pub global activate flutterfire_cli
flutterfire configure
flutter run -d chrome
```

Estructura sugerida:

```
lib/
├── main.dart
├── firebase_options.dart
├── models/
│   ├── user.dart
│   ├── pet.dart
│   ├── post.dart
│   └── comment.dart
├── services/
│   ├── auth_service.dart
│   ├── pet_service.dart
│   ├── post_service.dart
│   └── notification_service.dart
├── providers/
│   ├── auth_provider.dart
│   └── pet_provider.dart
├── screens/
│   ├── splash_screen.dart
│   ├── login_screen.dart
│   ├── register_screen.dart
│   ├── home_screen.dart
```

```
│   ├── profile_screen.dart
│   ├── adoption_screen.dart
│   ├── pet_detail_screen.dart
│   ├── create_post_screen.dart
│   └── chat_screen.dart
├── widgets/
│   ├── post_card.dart
│   ├── pet_card.dart
│   ├── stories_carousel.dart
│   └── like_button.dart
└── utils/
    ├── constants.dart
    └── validators.dart
```

Criterios de Evaluación.

Evaluación Continua (60% – Durante la clase).

- **Conexión y participación:** Estar en Live Share, interactuar, hacer preguntas.
- **Avance parcial:** Antes de finalizar la clase, subir un `.zip` del proyecto (excluyendo `build`, `.dart_tool`, `node_modules`) o enlace a repositorio GitHub/Drive con el avance. Debe incluir:

- Proyecto Flutter Web corriendo en Chrome.
- Firebase configurado (Auth funcionando).
- Al menos un modelo y un servicio implementados.
- Feed básico mostrando datos de Firestore.

- **Calidad de código:** Estructura modular, uso de Provider, nombres claros, comentarios en partes complejas.

Evaluación Final (40% – Entrega hasta el martes siguiente).

- **Funcionalidad:** Cumplir con al menos 8 de los 14 requisitos (PV1 a PV14).

- **Firebase:** Auth, Firestore y Storage funcionando correctamente.
- **Tiempo real:** Los likes y comentarios se actualizan sin recargar.
- **PWA:** Instalable en móvil/desktop, con manifest y service worker.
- **Notificaciones:** Al menos una notificación push funcionando.
- **Despliegue:** App publicada en Vercel/Netlify/Firebase Hosting (URL funcional).
- **Informe técnico:** Documento con arquitectura, decisiones, problemas y soluciones.
- **Código limpio:** Uso de Dart idiomático, null-safety, providers bien estructurados.

Entregables.

Durante la clase (Avance Parcial).

1. Enlace de Live Share: Compartir en el comentario privado de la plataforma antes de empezar a codificar.
2. Avance Parcial: Subir a la plataforma:

- `.zip` del proyecto (sin `build/`, `.dart_tool/`, `node_modules/`), O
- Enlace a repositorio GitHub con el avance, O
- Enlace a Google Drive con el archivo comprimido.

3. Capturas de pantalla del avance (login, feed, etc.).

Entrega Final (Martes siguiente).

1. Repositorio GitHub: Repositorio público con el código completo y `README.md` bien documentado.
2. App desplegada: URL funcional en Vercel/Netlify/Firebase Hosting.
3. Backend desplegado: Cloud Functions desplegadas (si aplica) o configuración Firebase visible.
4. Informe Técnico: En PDF o Markdown, que incluya:

- Introducción y objetivos.

- Explicación de la arquitectura (modelos, servicios, providers, screens).
- Modelo de datos (colecciones Firestore, Storage, Auth).
- Capturas de pantalla de cada funcionalidad.
- Dificultades encontradas y soluciones.
- Conclusiones y aprendizajes.
- Enlace al despliegue y al repositorio.

5. Video demo: Grabación de 2-3 minutos mostrando la PWA instalada y funcionando.

Deben subir ambas versiones: el avance parcial de la clase y la versión final.

Sistema Integrador y Defensa Final.

Recordatorio clave: PetVerse no es un proyecto aislado. Forma parte del Sistema integrador del segundo cuatrimestre, donde deberán combinar todas las tecnologías vistas durante el año (frontend web, frameworks mobile, backend, bases de datos, cloud, desktop, IA) en un proyecto unificado, o bien acoplarlo a su proyecto final.
Defensa final: Al final del cuatrimestre, defenderán su proyecto final explicando:

- La arquitectura y decisiones técnicas.
- Cómo se integran las diferentes tecnologías.
- El código (cada línea importante).
- Los desafíos y cómo los resolvieron.

Enfoque en código:

- Usar null-safety en Dart (obligatorio).
- Separar modelos, servicios y UI (arquitectura limpia).
- Usar Provider/Riverpod para state management.
- Comentar funciones complejas (especialmente streams de Firestore).
- Mantener un historial de commits claro y descriptivo.
- Escribir código modular, reutilizable y testeable.
- Documentar con `///` docstrings las clases y funciones públicas.